

Build Phase 3 — Playwright Deepening (1.5 hours)

Goal

You have a working app, a passing happy-path test, and at least a negative-path test started (or passing) from Phase 2. Phase 3 is about **finishing that foundation and going deeper** — one additional richer scenario, selector quality improvements, and a polished, rehearsed demo you're proud to show.

Step 1: Plan and Generate Your Prompts (10 min)

Same workflow as before — follow the **Copilot Workflow Guide** (`copilot-workflow-guide.md`):

1. Use **Plan agent** to describe which additional Playwright scenarios you want to automate
2. **Agent Mode** to break it into prompts
3. Have Copilot save them to `phase3-prompts.md`

Step 2: Finish or Expand Your Negative-Path Tests (20-25 min)

You should have at least one negative-path test started from Phase 2. Finish it here if it isn't passing yet, then add a second failure case if time allows.

If your Phase 2 negative-path test is passing: Add a second failure scenario — a different invalid field, a boundary-value case, or a business-rule failure specific to your document type.

If it's still failing: Fix it now. Focus on getting one clean, reliable assertion before adding more tests.

What each negative-path test should verify:

- The form submits or blocks as expected
- A clear error state is visible
- The exact validation message appears (not just "page contains some text")
- The success state does not appear

***Copilot tip:** "Write a Playwright test for an invalid [document type] submission where [field] is missing. Assert that the exact validation error appears and the success banner is not shown."*

Step 3: Add One Deeper Scenario (25-30 min)

Choose one richer workflow that makes your test suite more realistic.

Good options:

- File upload flow
- Alternate document type with different rules
- Backend failure or retry handling
- Mocked API response for a specific edge case

Pick **one** and finish it well.

***Copilot tip:** *Suggest one deeper Playwright scenario for this app that is realistic but still achievable in this session, then generate the test and any small app changes needed to support it.**

Step 4: Improve Selectors and Assertions (15-20 min)

Review your tests and make them more stable and readable.

****What to improve:****

- Replace brittle selectors with ``getByRole``, ``getByLabel``, or clearer text-based locators
- Make assertions more specific
- Remove unnecessary waits or fragile timing assumptions

***Copilot tip:** *Review this Playwright spec and replace brittle selectors with more stable accessible locators. Explain why each change improves reliability.**

Step 5: Debug a Failure with Trace or Screenshot Output (15-20 min)

Use Playwright's debugging tools as part of the learning experience.

****What to do:****

- Intentionally run a failing test or inspect a real failure
- Open the trace or screenshot output
- Ask Copilot to help you interpret what happened
- Fix the issue and re-run

***Copilot tip:** *This Playwright test failed. Based on the error and trace, help me understand what happened and how to make the test more reliable.**

Step 6: Prepare and Rehearse Your Demo (10-15 min)

Don't skip this. A polished 5-minute demo is worth more than a half-finished extra test.

****What to do:****

- Stop adding new tests. Get everything in a passing state.
- Decide what you'll show: which user flow, which test output, which Copilot interaction
- Prepare test data so your demo doesn't rely on luck
- Do a full app restart and verify it runs clean from scratch
- Rehearse your 5-minute story:
 1. What does the app do?
 2. Show one passing test and one failure test
 3. What was the most useful Copilot moment?
 4. What surprised you or what would you do differently?

***Copilot tip:** *Help me write a 5-minute demo script for a hackathon presentation. My app [describe it]. I have these Playwright tests [describe them]. What's the cleanest story to tell?**

Step 7: Create Project Documentation (10-15 min)

Before final demos, create three markdown files so someone new can run and understand your project quickly.

1. ``README.md`` (overall project overview)
 - What the app does and which document type it validates
 - How to run the app locally
 - Where the API and frontend live
 - Quick summary of Playwright coverage
2. ``TESTING.md`` (how to test)
 - Playwright install/setup command(s)
 - How to run all tests and a single test
 - Test folder structure and fixture location (``test-data/``)
 - Common test failure tips (selector issues, timing, local server not running)
3. ``techOverview.md`` (technical details)
 - Stack and key libraries
 - High-level architecture (frontend -> API -> validation)
 - Validation rules implemented
 - Known limitations and next improvements

Keep these docs concise and practical. Aim for clear run instructions someone can follow in under 10 minutes.

***Copilot tip:** "Generate `README.md`, `TESTING.md`, and `techOverview.md` for this project. Keep each file concise and include exact run/test commands for my stack."*

Optional Stretch Goals

If you finish early, choose one:

- Add unit tests for your validation logic
- Support multiple document types in the app
- Accept PDF uploads
- Build a dashboard or persistence layer
- Create Copilot customization files such as ``.github/copilot-instructions.md``, ``SKILL.md``, or ``.agent.md``

Checkpoint

By the end of Phase 3:

- ☐ Negative-path test from Phase 2 is passing (or expanded to a second case)
- ☐ One deeper scenario or debugging exercise completed
- ☐ Selectors and assertions reviewed and tightened
- ☐ Demo rehearsed — you know your 5-minute story
- ☐ Documentation created: ``README.md``, ``TESTING.md``, and ``techOverview.md``
- ☐ App runs clean from a fresh start

****If you're ahead of schedule:**** Pick an optional stretch goal or strengthen your assertions further.

****If you're behind:**** Skip additional tests. Get what you have passing, rehearse the demo, and be ready to present what works.